

1. Allocator 설계 구조

단순한 implicit/explicit free list 방식이 가진 성능적 한계를 극복하고, throughput과 utilization를 극대화하기 위해 segregated free list 방식을 기반으로 설계하였습니다.

핵심 설계 원리

1) Size Classes - 크기별 분류

free block들을 크기에 따라 20개의 서로 다른 이중 연결 리스트로 분리하여 관리합니다. 특정 크기의 메모리 할당 요청이 들어오면 해당 크기 범위를 담당하는 리스트만 빠르게 탐색하는 것으로 탐색 시간을 줄일 수 있습니다.

2) First-Fit Search

알맞은 크기의 리스트를 찾으면, 리스트의 처음부터 순회하며 요청 크기 이상인 블록 중 첫 번째로 만나는 블록을 즉시 선택하여 탐색 비용을 최소화했습니다.

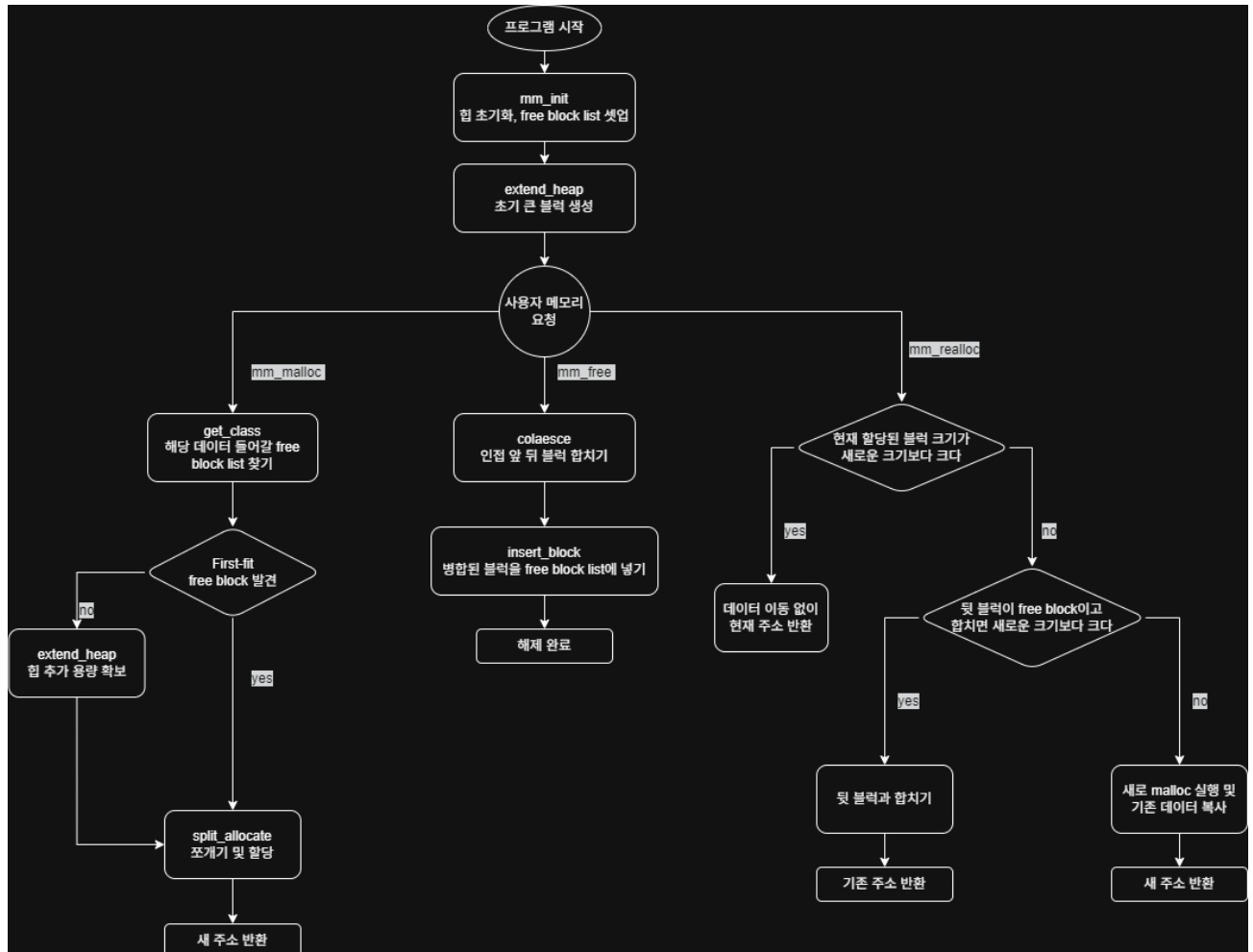
3) Splitting & Coalescing

Splitting: 큰 블록을 할당할 때 필요한 만큼만 잘라내고(최소 블록 크기 24바이트 보장), 남은 공간은 새로운 free block으로 만들어 리스트에 되돌려놓아 internal fragmentation를 방지합니다.

Coalescing: 블록 free 시 전후로 인접한 free block이 있다면 즉시 하나의 거대한 free block으로 병합하여 External Fragmentation를 억제합니다.

2. 다이어그램 및 아키텍처

1) 전체 코드 실행 흐름



2) 개별 블록 구조

모든 메모리 블록이 Alignment 조건을 준수하도록 설정하였습니다.

Allocated Block

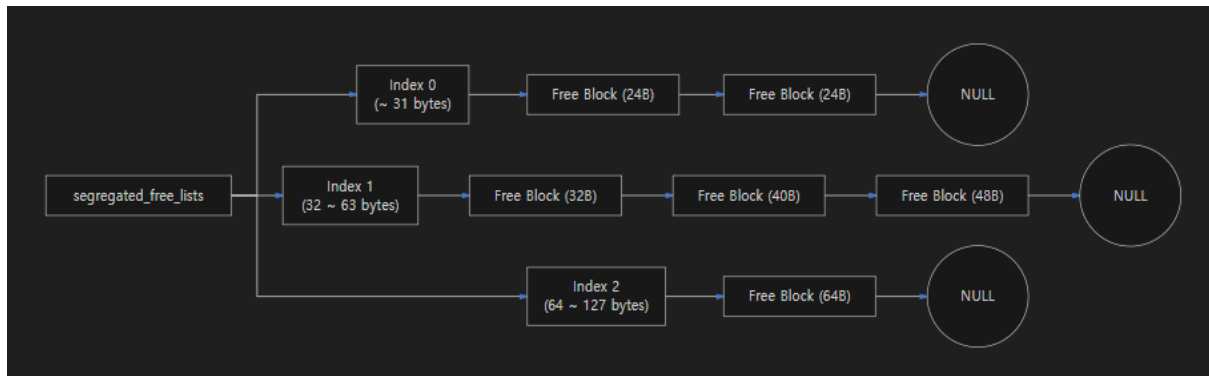
구성 요소	크기 (Bytes)	설명
Header	4	전체 크기 및 할당 상태(1) 저장
Payload	변동	실제 사용자 데이터 영역
Padding	변동	8바이트 정렬을 위한 여백 (필요 시)
Footer	4	Header와 동일한 정보 저장

Free Block

구성 요소	크기 (Bytes)	설명
Header	4	전체 크기 및 할당 상태(0) 저장
Prev Pointer	8	이전 free block의 메모리 주소
Next Pointer	8	다음 free block의 메모리 주소
(Empty Space	변동	사용되지 않는 잉여 빈 공간)
Footer	4	Header와 동일한 정보 저장

3) Segregated List Array 아키텍처

각 인덱스는 2의 거듭제곱 크기 범위에 해당하는 free block들의 이중 연결 리스트 헤드를 가리킵니다. 0번 인덱스는 31바이트 이하의 블록, 1번은 32~63바이트, 2번은 64~127바이트 크기의 블록을 담당하는 식으로 수용 범위가 2배씩 증가합니다. 마지막 19번 인덱스는 이 규칙을 초과하는 모든 초대형 블록(약 16MB 이상)들을 통틀어서 관리합니다.



3. 전역 변수

```
static void **segregated_free_lists;
```

20개의 크기별 free block list들의 헤드 포인터를 저장하는 배열의 시작 주소를 가리킵니다.

메모리 할당 및 해제 시, 특정 크기의 블록이 저장된 리스트에 빠르게 접근하기 위한 진입점 역할을 합니다.

```
static char *heap_listp;
```

힙 메모리의 초기 기준점 역할을 하는 포인터입니다.

프로로그 헤더와 푸터 사이를 정확히 가리키도록 초기화되며, Coalesce 시 힙의 맨 앞 경계를 벗어나는 것을 방지하는 방어벽 역할을 수행합니다.

4. 추가 함수 설명

static int get_class(size_t size)

역할: 주어진 블록 크기가 20개의 Segregated Free List 중 몇 번째 인덱스에 해당하는지 계산하여 반환합니다.

static void *extend_heap(size_t words)

역할: 힙 영역에 사용 가능한 메모리가 부족할 때, 운영체제(mem_sbrk)에 요청하여 지정된 words 개수만큼의 거대한 free block을 새롭게 생성합니다.

static void split_allocate(void *bp, size_t target_block_size)

역할: 탐색된 충분히 큰 free block 내부에 요청받은 target_block_size만큼의 메모리를 할당 상태로 만듭니다. 만약 전체 크기에서 목표 크기를 빼고 남은 여유 공간이 최소 free block 크기(24바이트) 이상이라면, 남은 공간을 분할하여 다시 free block list에 삽입함으로써 내부 단편화를 줄입니다.

static void *coalesce(void *bp)

역할: 특정 블록이 free되거나 힙이 확장되었을 때, 해당 블록의 앞이나 뒤에 이미 비어있는 인접 free block이 있다면 이들을 하나의 큰 free block으로 하나로 병합합니다.

static void insert_node(void *bp, size_t size)

역할: 병합되거나 새롭게 생성된 free block을 크기에 맞는 Segregated List 내부의 알맞은 위치에 삽입합니다. 외부 단편화를 줄이기 위해 주소 오름차순 정렬 삽입 로직을 적용했습니다.

static void delete_node(void *bp)

역할: 할당되거나 병합 대상이 되어 더 이상 free block이 아니게 된 블록을 Segregated List 구조 내에서 연결 포인터를 수정하여 안전하게 제거합니다.

5. 핵심 인터페이스 함수 설명

int mm_init(void)

메모리 할당기 시스템을 최초로 초기화합니다.

20개의 Segregated Free List 헤드 포인터를 담을 배열 공간을 힙의 맨 앞에 할당하고 모두 NULL로 초기화합니다. 이후 프로로그 블록과 에필로그 블록을 생성하여 경계를 설정하고, extend_heap을 호출해 초기 free block을 확보합니다. 초기화 성공 시 0, 실패 시 -1을 반환합니다.

void *mm_malloc(size_t size)

사용자로부터 요청받은 size 바이트만큼의 메모리 공간을 할당합니다.

요청된 크기에 헤더와 푸터 크기(8바이트)를 더하고 8바이트 정렬 조건에 맞춰 실제 필요한 전체 블록 크기를 교정합니다. 이후 get_class와 First-Fit 방식을 통해 적절한 free block을 찾고, split_allocate를 통해 메모리를 할당한 뒤 해당 payload의 시작 주소를 반환합니다. 넉넉한 free block이 없다면 extend_heap으로 힙을 확장합니다.

void mm_free(void *bp)

이전에 할당되었던 메모리 블록을 다시 시스템에 free 상태로 반환합니다.

인자로 받은 포인터 bp의 헤더와 푸터의 할당 상태 비트를 0(Free)으로 변경합니다. 그 직후 coalesce 함수를 호출하여 앞뒤로 인접해 있는 free block이 있다면 하나로 병합하고, 새롭게 만들어진 큰 free block을 알맞은 리스트에 삽입합니다.

void *mm_realloc(void *ptr, size_t size)

이미 할당된 메모리 블록의 크기를 새로운 size로 변경합니다. 사이즈가 0이면 mm_free를, 포인터가 NULL이면 mm_malloc을 수행합니다.

확장이 필요할 때, 바로 뒷 블록이 비어있고 두 블록을 합쳤을 때 요청 크기를 넉넉하게 만족한다면, 새로 메모리를 할당해서 값을 옮기지 않습니다. 단순히 뒷 블록을 허물어 하나로 합친 뒤 기존 포인터를 그대로 반환합니다.

뒷 블록이 이미 존재한다면 어쩔 수 없이 새 블록을 mm_malloc으로 구하고 데이터를 복사한 뒤, 기존 방을 해제합니다.